МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине: «Безопасность систем баз данных»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2025г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2025г

_____

(подпись)

Новосибирск 2025

Содержание

Введение 3
1. ER-модель и классификация 4
2. DDL и схемы 9
3. Роли и привилегии 11
4. Логирование подключений пользователей 13
Тестирование 14
Развертывание 18
Вывод 19
Приложение 20

Введение

Цель работы: спроектировать доменную схему, классифицировать данные и настроить доступ с разделением обязанностей.

Тема: «Защита БД студии дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниями записей и защищённым архивам аудиофайлов».

Задачи:

1. ER-модель и классификация: сформировать ER диаграмму домена с указанием ключей и связей. Выполнить классификацию данных. Сформировать матрицу «колонка – класс».

2. DDL и схемы: создать схемы: app, ref, audit, stg. Реализовать DDL (PK, FK, UNIQUE, CHECK, индексы, NOT NULL). Сгенерировать тестовые данные.

3. Роли и привилегии: спроектировать и создать роли: app_reader, app_writer, app_owner, auditor; и роли разделения обязанностей: ddl_admin, dml_admin, security_admin. Назначить привилегии на уровнях схем/таблиц/последовательностей. Настроить default privileges и запретить public доступ. Подготовить сценарий проверки: какие роли что видят/могут изменять.

4. Логирование подключений пользователей: создать таблицу audit.login_log (login_time, username, client_ip) и триггер или функцию, которая при подключении пользователя записывает информацию о входе и протестировать под разными ролями.

1. ER-модель и классификация

Сначала определим все сущности и их определение, необходимые для реализации базы данных по студии дубляжа в таблице 1.

Таблица 1 – Сущность и определение сущности

Наименование сущности	**Определение сущности**
actors	Хранит данные актёров, в том числе ФИО, номер телефона, электронную почту
audio_archives	Хранит путь до аудиофайлов, размер, длину, а также содержит ссылку на сессию
contracts	Содержит информацию о контрактах с актерами, а также ссылки на актёра и проект
projects	Содержит информацию о проектах, в том числе название, язык проекта и ссылку на студию
recording_sessions	Хранит все данные о планируемых сессиях и содержит ссылку на контракт
voice_roles	Содержит информацию о ролях, а также ссылки на проект, актера и язык
login_log	Содержит информацию о входах в систему и подключениях к базе данных
languages	Содержит языки, которые используются в проектах
studios	Содержит студии, которые используются в проектах

На основе определенных сущностей составим ER-диаграмму домена, где мы уточнили атрибуты каждой отдельной сущности (Рисунок 1).

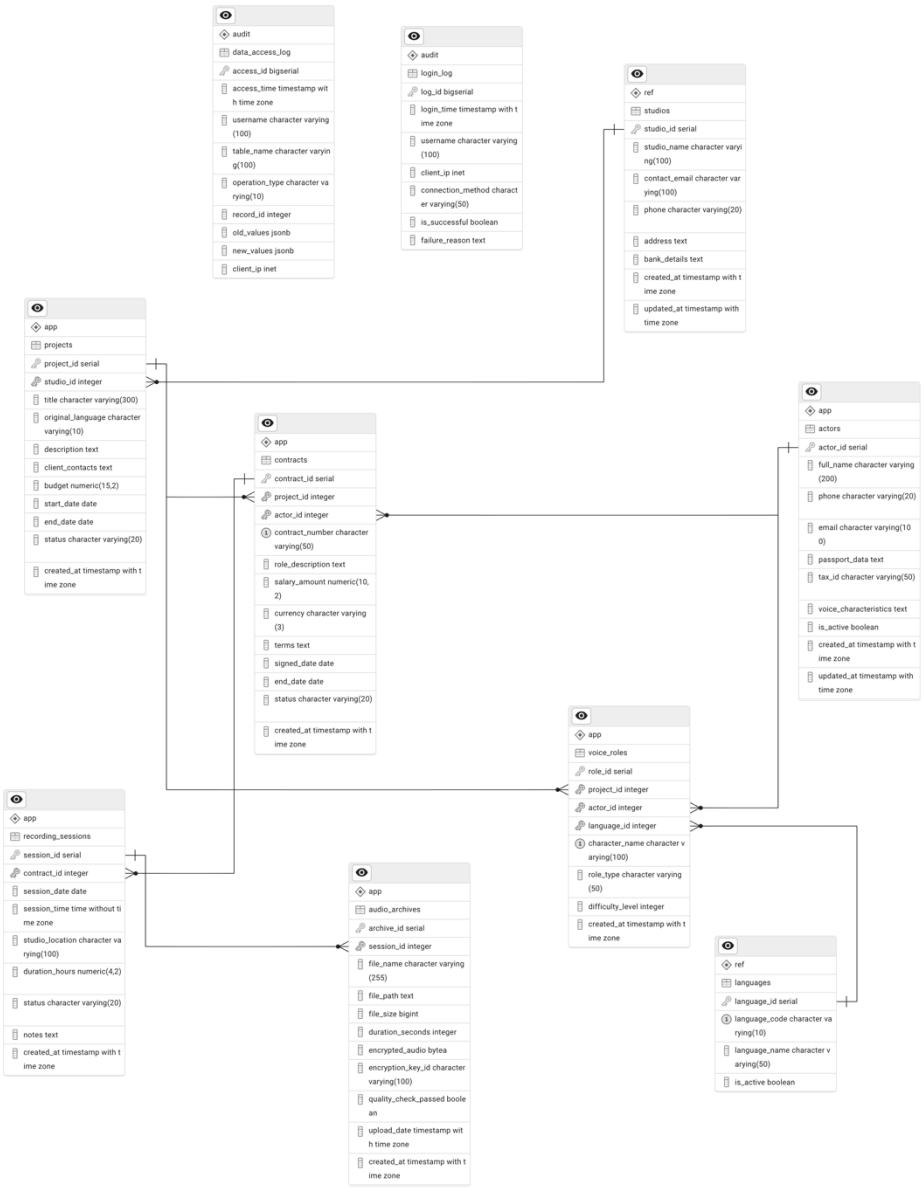


Рисунок 1 – ER-диаграмма домена

Далее будет произведена классификация данных на следующих уровнях:

1. Public – публичные данные;
2. Internal – внутренняя информация учреждения, которая не подлежит свободному распространению;
3. Confidential – данные доступные только ограниченному кругу лиц;

4. Restricted – конфиденциальные данные.

Классифицированные данные представлены в таблицах 2–10 в формате матриц вида «колонка → класс».

Таблица 2 – Классификация данных таблицы studios

Колонка	**Класс**
studio_id	Internal
studio_name	Public
contact_email	Internal
phone	Internal
address	Internal

Таблица 3 – Классификация данных таблицы languages

Колонка	**Класс**
language_id	Internal
language_code	Public
language_name	Public
is_active	Internal

Таблица 4 – Классификация данных таблицы actors

Колонка	**Класс**
actor_id	Internal
full_name	Public
phone	Restricted
email	Restricted

Таблица 5 – Классификация данных таблицы projects

Колонка	**Класс**
project_id	Internal
studio_id	Internal
title	Public
original_language	Public

Таблица 6 – Классификация данных таблицы contracts

Колонка	**Класс**
contract_id	Internal
project_id	Internal
actor_id	Internal
contract_number	Internal
role_description	Internal
salary_amount	Confidential
currency	Internal
terms	Confidential
signed_date	Internal

Таблица 7 – Классификация данных таблицы recording_sessions

Колонка	**Класс**
session_id	Internal
contract_id	Internal
session_date	Internal
session_time	Internal
studio_location	Internal
duration_hours	Internal
status	Internal
notes	Internal

Таблица 8 – Классификация данных таблицы audio_archives

Колонка	**Класс**
archive_id	Internal
session_id	Internal
file_name	Internal
file_path	Internal

Таблица 9 – Классификация данных таблицы voice_roles

Колонка	**Класс**
role_id	Internal
project_id	Internal
actor_id	Internal

Таблица 10 – Классификация данных таблицы login_log

Колонка	**Класс**
log_id	Internal
login_time	Confidential
username	Confidential
client_ip	Restricted
connection_method	Internal
is_successful	Internal
failure_reason	Internal

2. DDL и схемы

В таблице 11 представлено соотношение схемы к таблицам.

Таблица 11 – Определение схем и таблиц

Схема	Таблица
app	actors
app	projects
app	contracts
app	recording_sessions
app	audio_archives
app	voice_roles
ref	studios
ref	languages
audit	login_log
stg	import_log

В организации первичных ключей используется стандартный подход с SERIAL для автоматической генерации идентификаторов.

Ограничения целостности включают UNIQUE-ограничения для защиты критически важных данных таких как номера контрактов и комбинации ролей, CHECK-ограничения обеспечивают валидацию данных включая уровень сложности ролей от 1 до 5, а внешние ключи поддерживают реляционную целостность между связанными сущностями.

Индексы созданы оптимизированные для часто используемых запросов включая индексы по внешним ключам для быстрых JOIN-операций, индексы по статусам для фильтрации активных записей и индексы по датам для временных запросов.

Безопасность данных обеспечивается через несколько механизмов: аудиофайлы хранятся в зашифрованном виде в поле encrypted_audio,

конфиденциальные поля помечены соответствующими комментариями, а также реализовано разделение доступа на уровне схем.

Также были сгенерированы тестовые данные (≥ 10 строк) в таких ключевых таблицах, как actors (актёры озвучивания), contracts (контракты с актёрами), recoring_sessions (сессии записи), audio_archives (аудиоархивы), voice_roles (роли озвучивания).

Скрипт создания схем и таблиц приведен в приложении в файле script2.sql.

3. Роли и привилегии

В таблице 13 представлено соотношение ролей и привилегий.

Таблица 13 – Соотношение роли и привилегий

Роль	Привилегии
app_reader	USAGE для схем app, ref; SELECT во всех таблицах схем app, ref
app_writer	USAGE для схем app, ref; SELECT, INSERT, UPDATE во всех таблицах схем app, ref
app_owner	USAGE для схем app, ref, stg; ALL PRIVILEGES на все объекты схем app, ref, stg
auditor	USAGE для схемы audit; SELECT во всех таблицах схемы audit
ddl_admin	USAGE для схем app, ref, audit, stg; CREATE для схем app, ref, audit, stg
dml_admin	USAGE для схем app, ref, stg; SELECT, INSERT, UPDATE, DELETE во всех таблицах схем app, ref, stg
security_admin	USAGE для всех схем; права ddl_admin и dml_admin; GRANT/REVOKE привилегий
voice_actor	USAGE для схем app, ref; SELECT в таблицах app и ref (только свои данные)

Также подготовлен скрипт со всеми сценариями проверки ролей в приложении в файле role_testing_scenarios.sql. Результат проверки сценариев представлен на рисунке 2.



Рисунок 2 – Проверка ролей

4. Логирование подключений пользователей

В нашей базе данных реализована система аудита. Функция audit.log_connection() автоматически записывает все подключения пользователей с фиксацией имени, IP-адреса и статуса авторизации (Рисунок 3).

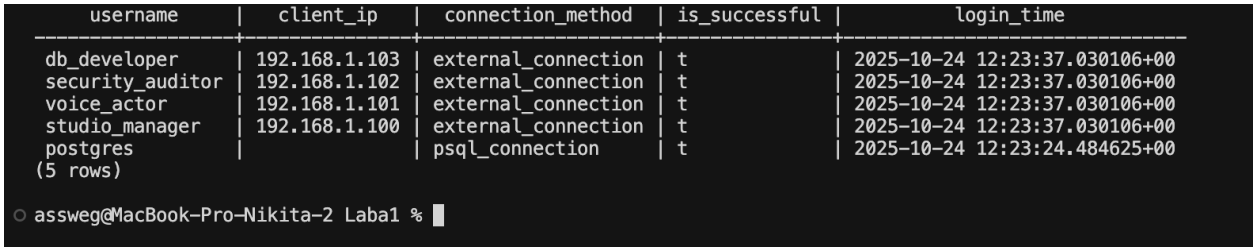


Рисунок 3 – Проверка аудита

Тестирование

Проведено тестирование роли app_reader на возможность SELECT таблицы app.actors и SELECT таблицы audit.login_log (Рисунок 4).

```
dubbing_studio=# SET ROLE app_reader;
SET
dubbing_studio=> SELECT COUNT(*) as actors_count FROM app.actors;
 actors_count
--------------
            0
(1 row)

dubbing_studio=> SELECT * FROM audit.login_log LIMIT 1;
ERROR:  permission denied for schema audit
LINE 1: SELECT * FROM audit.login_log LIMIT 1;
                      ^
dubbing_studio=>
```

Рисунок 4 – Тестирование роли app_reader

Проведено тестирование роли app_writer на возможность UPDATE таблицы app.actors, DELETE строки в таблице app.actors и SELECT таблицы audit.login_log (Рисунок 5).

```
dubbing_studio=> SET ROLE app_writer;
SET
dubbing_studio=> UPDATE app.actors SET phone = '+7-888-888-8888'
WHERE full_name = 'Test Writer Actor'
RETURNING actor_id, full_name, phone;
 actor_id | full_name | phone
----------+-----------+-------
(0 rows)

UPDATE 0
dubbing_studio=> DELETE FROM app.actors WHERE full_name = 'Test Writer Actor';
ERROR:  permission denied for table actors
dubbing_studio=> SELECT * FROM audit.login_log LIMIT 1;
ERROR:  permission denied for schema audit
LINE 1: SELECT * FROM audit.login_log LIMIT 1;
                      ^
dubbing_studio=>
```

Рисунок 5 – Тестирование роли app_writer

Проведено тестирование роли studio_manager на возможность SELECT, UPDATE строки в таблице app.actors и SELECT таблицы audit.login_log (Рисунок 6).

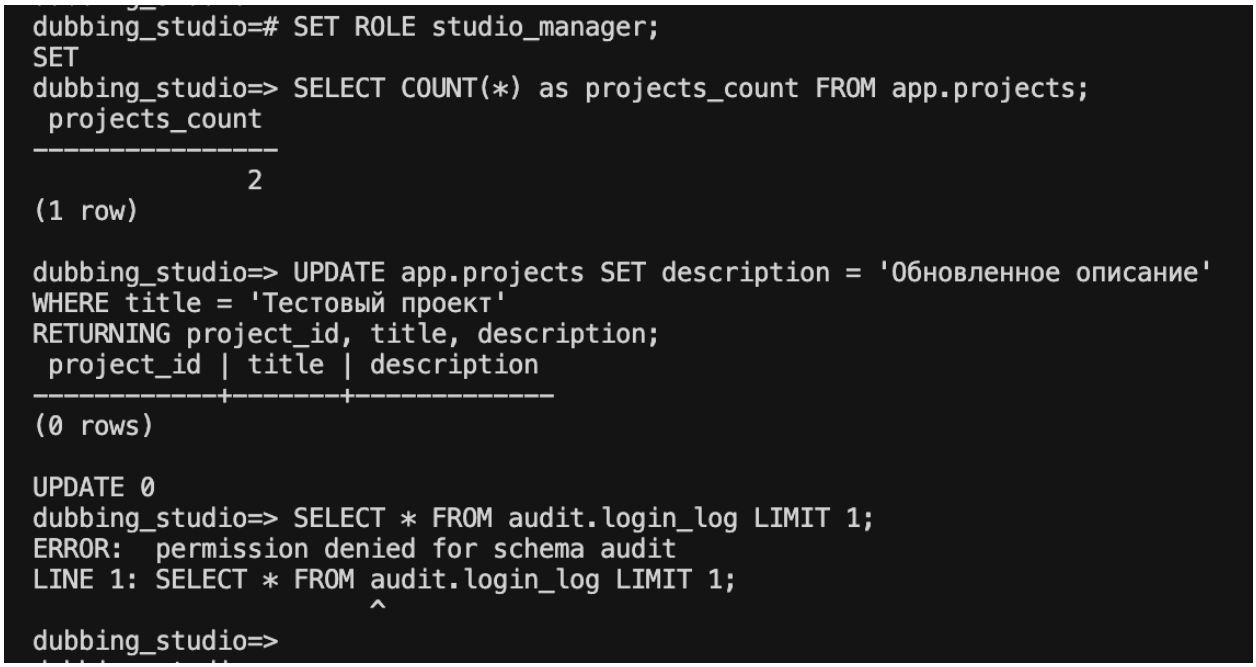


Рисунок 6 – Тестирование роли studio_manager

Проведено тестирование роли voice_actor на возможность SELECT таблицы app.contracts, SELECT функции и INSERT в таблицу app.recording_sessions (Рисунок 7).

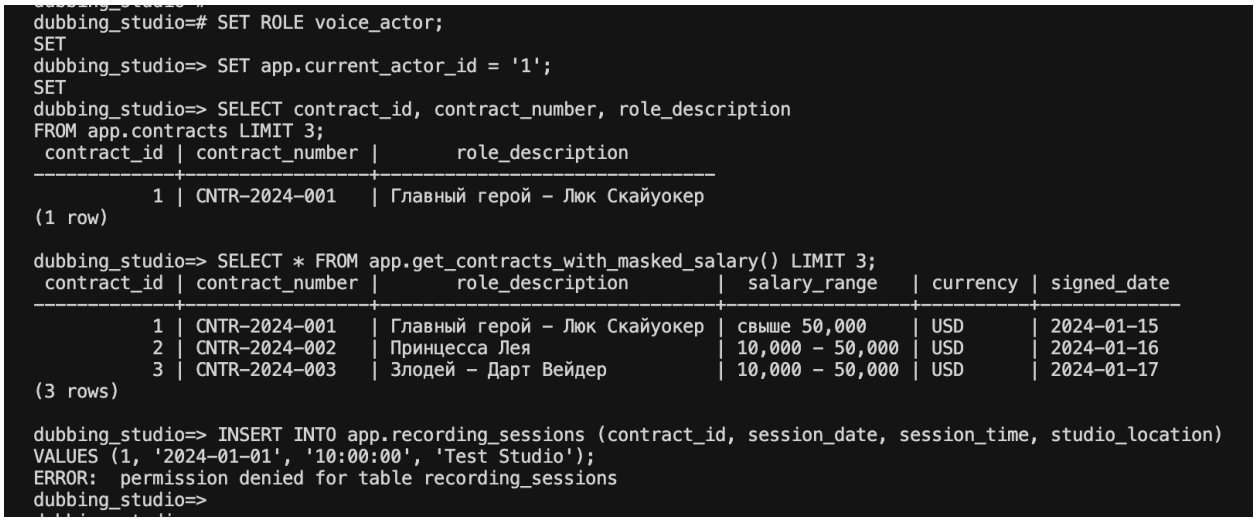


Рисунок 7 – Тестирование роли voice_actor

Проведено тестирование роли security_auditor на возможность SELECT и SELECT таблицы app.contracts (Рисунок 8).

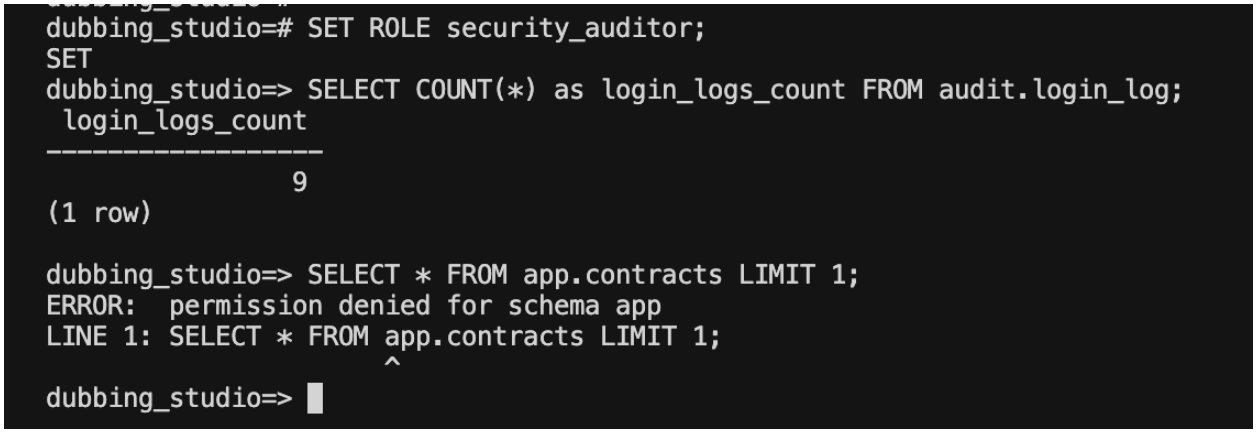


Рисунок 8 – Тестирование роли security_auditor

Проведено тестирование роли app_owner на возможность SELECT, GRANT и UPDATE (Рисунок 9).

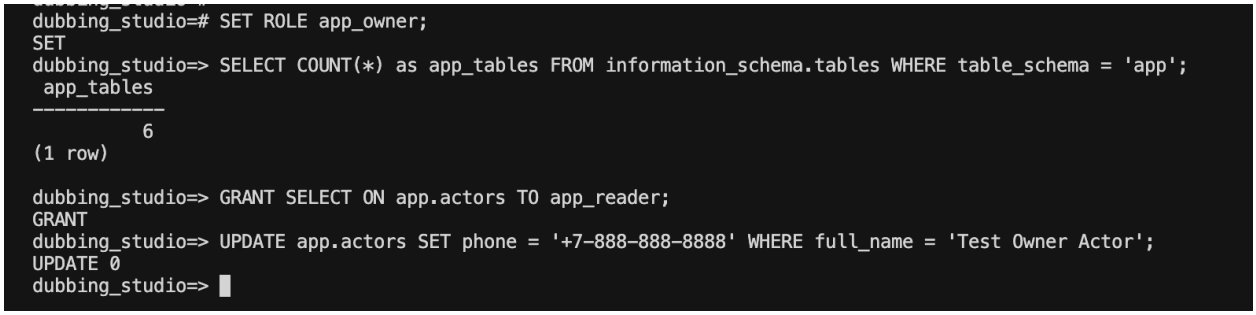


Рисунок 9 – Тестирование роли app_owner

Проведено тестирование роли dml_admin на возможность массового INSERT в таблицу app.actors и массовый DELETE таблицы app.actors (Рисунок 10).

```
dubbing_studio=# SET ROLE dml_admin;
SET
dubbing_studio=> INSERT INTO app.actors (full_name, phone) VALUES
('DML Admin Actor 1', '+7-111-111-1111'),
('DML Admin Actor 2', '+7-222-222-2222')
RETURNING actor_id, full_name;
 actor_id |     full_name
----------+-------------------
       11 | DML Admin Actor 1
       12 | DML Admin Actor 2
(2 rows)

INSERT 0 2
dubbing_studio=> UPDATE app.actors SET phone = '+7-999-999-9999'
WHERE full_name LIKE 'DML Admin Actor%'
RETURNING actor_id, full_name, phone;
 actor_id |     full_name     |      phone
----------+-------------------+-----------------
       11 | DML Admin Actor 1 | +7-999-999-9999
       12 | DML Admin Actor 2 | +7-999-999-9999
(2 rows)

UPDATE 2
dubbing_studio=>
```

Рисунок 10 – Тестирование роли dml_admin

Проведено тестирование роли ddl_admin на возможность создания таблицы, создания индекса, удаления индекса и удаления таблицы (Рисунок 11).

```
dubbing_studio=# SET ROLE ddl_admin;
SET
dubbing_studio=> CREATE TABLE app.test_ddl_table (
    id SERIAL PRIMARY KEY,
    data TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE
dubbing_studio=> CREATE INDEX idx_test_ddl_data ON app.test_ddl_table(data);
CREATE INDEX
dubbing_studio=> DROP INDEX idx_test_ddl_data;
ERROR:  index "idx_test_ddl_data" does not exist
dubbing_studio=> DROP TABLE app.test_ddl_table
dubbing_studio->
```

Рисунок 11 – Тестирование роли ddl_admin

Развертывание

База данных была развернута с помощью докера. Для того, чтобы развернуть нашу систему сначала нужно скачать проект с гитхаба с помощью команды: git clone https://github.com/Assweg/laba1_bsbd.git.

Далее переходим в папку с проектом и открываем терминал. Чтобы развернуть и инициализировать базу данных пишем команду: ./run_docker.sh start. Для подключения к базе данных используем команду: ./run_docker.sh connect.

На рисунке 12 представлен вывод после команды ./run_docker.sh start.

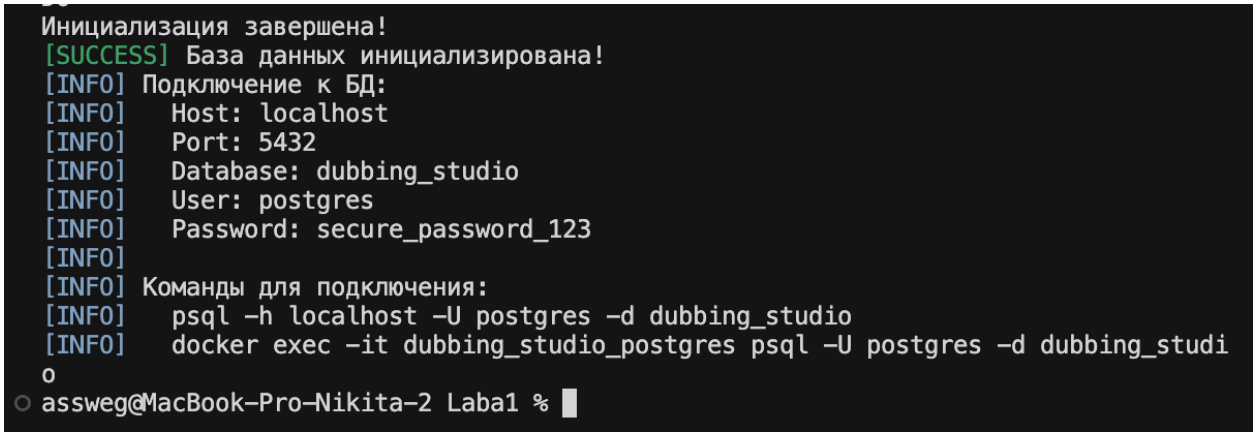


Рисунок 12 – Запуск базы данных

На рисунке 13 представлено подключение к базе данных.

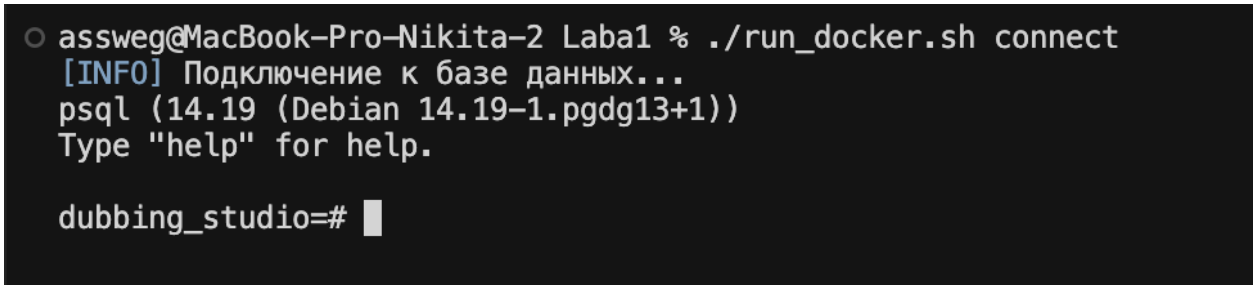


Рисунок 13 – Подключение к базе данных

Вывод

В ходе лабораторной работы была создана ER-диаграмма домена из десяти таблиц с всеми связями. Выполнена классификация данных на четыре уровня безопасности от Public до Restricted.

Также реализованы четыре схемы app, ref, audit, stg с полным набором ограничений включая первичные и внешние ключи, уникальные ограничения, проверочные условия, индексы производительности и обязательные поля. Создано более десяти записей тестовых данных в каждой ключевой таблице.

Спроектированы и реализованы семь ролей включая app_reader, app_writer, app_owner, auditor и роли разделения обязанностей ddl_admin, dml_admin, security_admin. Настроены привилегии на уровне схем, таблиц и последовательностей с полным запретом PUBLIC доступа и настройкой DEFAULT PRIVILEGES для новых объектов.

Была создана таблица audit.login_log с полями login_time, username, client_ip и реализованы SECURITY DEFINER функции для автоматического и ручного логирования подключений пользователей. Настроены триггеры для аудита DML операций.

Приложение

Ссылка на GitHub-репозиторий:

https://github.com/Assweg/laba1_bsbd.git